YEAR 1 · TERM 1 · MODULE 3 · WEEK 1

Introduction to Software Engineering Management

Software is built by teams, changes constantly, and must be traceable. This week you learn how development is organised as a process, how configuration management keeps the product under control, and how Git and GitHub let people work on the same code without losing each other's changes.

Learning objectives

- Explain the difference between programming and software engineering.
- Describe a software development lifecycle and two process models.
- Write checkable requirements and tell functional from non-functional ones.
- Read and sketch basic models: context diagram, ER model, state machine.
- Define software configuration management and its main activities.
- Use basic Git commands to initialise, stage, commit, branch, and merge.
- Connect a local repository to a remote and describe a pull request workflow.

Agenda

01

What software engineering is

02

The development lifecycle and process models

03

Requirements engineering: from wishes to specifications

04

Modelling software: from context diagrams to UML

05

Software configuration management

06

Version control systems

07

Git fundamentals: areas, commits, and branches

08

GitHub, remotes, and collaboration workflows

LESSON 1

What software engineering is

Key takeaways

- Software engineering is a discipline about delivering and maintaining software, not only writing it.
- Most project problems come from communication and planning gaps.
- Models let teams discuss and check a system before building it.

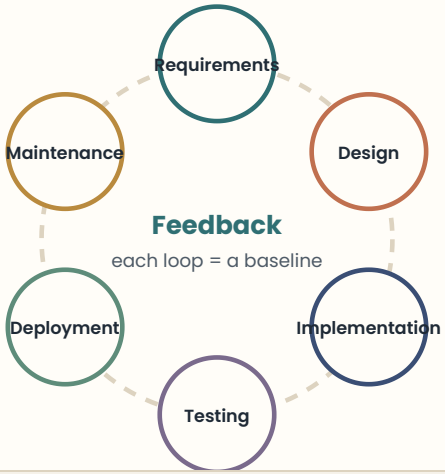
> **Think of it like this** Building a house needs blueprints before bricks. Software needs plans and models before code, because changing a wall on paper costs far less than tearing it down.

LESSON 2

The development lifecycle and process models

Key takeaways

- A process model is a simplified description of how real teams work.
- Iterative work produces baselines; the difference between baselines is an increment.
- Choose a process by how much change you expect and how costly late mistakes are.



Think of it like this Think of a cooking class. Waterfall is following a recipe from start to finish without tasting. Iterative work is tasting the sauce after each step and adjusting the next one.

LESSON 3

Requirements engineering: from wishes to specifications

Key takeaways

- A requirement must be checkable: someone should be able to say "met" or "not met".
- Seven activities: inception, elicitation, elaboration, negotiation, specification, validation, management.
- Functional = what the system does. Non-functional = how well it does it.



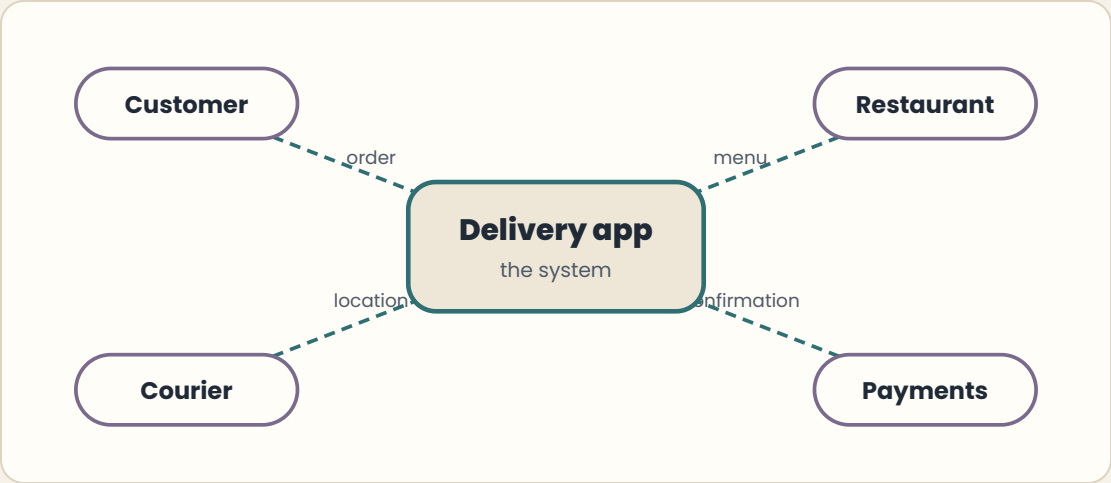
Think of it like this A tailor measures twice and cuts once. Requirements are the measurements: if they are wrong, no amount of careful sewing (coding) will make the suit fit.

LESSON 4

Modelling software: from context diagrams to UML

Key takeaways

- A model is useful when it is simpler than the system but still correct about what matters.
- Context diagram = boundaries; ER model = data; state machine = lifecycle of one thing.
- UML has structure diagrams and behaviour diagrams; choose by the question you need answered.



Think of it like this A city map is a model. It ignores the colour of each house (abstract) but gets the streets right (precise), so you can plan a route before walking (predictive and cheap).

LESSON 5

Software configuration management

Key takeaways

- Configuration items include everything that makes up the product, not only code.
- SCM answers: which version, which change, approved by whom, and in which baseline.

Think of it like this A library keeps a catalogue of every edition and who borrowed it. SCM does the same for a product: every item, version, and change has a record you can check.

LESSON 6

Version control systems

Key takeaways

- A VCS stores history, supports recovery, and shows who changed what and when.
- Git is distributed; GitHub is a hosting service for Git repositories.
- One commit should hold one logical change with a clear message.

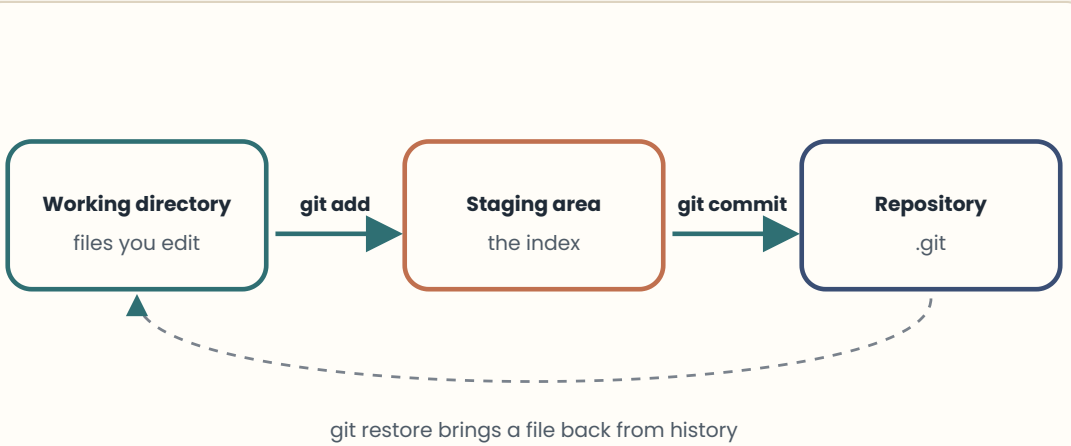
Think of it like this Version control is like a video game with save points. You can go back to any save, see what changed between two saves, and let friends play from their own copy.

LESSON 7

Git fundamentals: areas, commits, and branches

Key takeaways

- Edit in the working directory, stage with git add, and record with git commit.
- git status and git log show where you are; use them before and after each step.
- A branch is a pointer. Merging combines history; conflicts require a human decision.



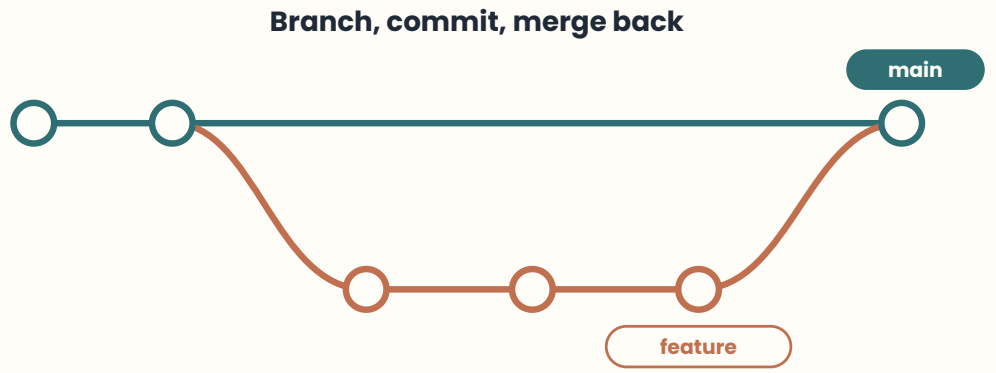
Think of it like this Git is a photo studio. The working directory is the set, staging is choosing which photos go in the album, and commit is printing the album page. A branch is a second album you can build without touching the first.

LESSON 8

GitHub, remotes, and collaboration workflows

Key takeaways

- origin is the conventional name for the remote you cloned or connected.
- fetch downloads; pull downloads and integrates; push uploads.
- Pull requests make review a planned step before code reaches main.



Each circle is a commit. A branch is a pointer. Merging joins histories.

Think of it like this A pull request is like a draft essay sent to a classmate before handing it in. The reviewer does not write it for you, but catches mistakes while they are cheap to fix.

Checkpoint

1. Which item is a configuration item in software configuration management?

A. A requirements document · B. The office coffee machine · C. A personal calendar

2. Which statement about Git is correct?

A. Git is a distributed version control system; every clone holds the full history. · B. Git cannot record a commit without an internet connection to GitHub. · C. Git and GitHub are two names for the same tool.

3. In an iterative and incremental process, what does the difference between two successive baselines represent?

A. An increment · B. A risk register · C. The final delivery only

Keep going

Review your flashcards when they are due, use the subject's tools, and retake the quiz until you score 80% or more.